sql/trip_positions_create_index_btree.sql
```
CREATE INDEX trip_positions_created_at_idx
ON trip_positions (created_at);

ANALYZE trip_positions;
```

With the B-Tree Index in place, the query runs much faster. The query execution plan shows an Index Only Scan using the "trip_positions_created_at_idx" index. Check the size of the index.

Running `SELECT pg_size_pretty(pg_relation_size('trip_positions_created_at_idx'));` in psql shows a size over 200 MB. Drop the Index and try creating a BRIN index instead.

sql/trip_positions_create_index_brin.sql
```
CREATE INDEX trip_positions_created_at_idx_brin
ON trip_positions USING brin (created_at);

ANALYZE trip_positions;
```

Run the same query again. You should see a Bitmap Index Scan using the "trip_positions_created_at_idx_brin" and very good query performance comparable to the B-Tree Index. Check the size of the BRIN index. The size is just around 40 kB. Much smaller!

You've seen how to use Indexes for filtering operations. How else can they be used?

Using Indexes In Less Common Ways

Besides speeding up queries, Indexes can be used for sort operations where `ORDER BY` clauses appear in a query. Indexes store sorted data as opposed to how the data might be ordered in pages.

> B-tree indexes are sorted in ascending order by default.[15]

In this section you'll put this concept into practice. For Rideshare the `trips` table has a `completed_at` column that's set when a trip is completed. The trip's `completed_at` is `NULL` when a trip is in progress.

Run the following query from psql to order the Trips by their completion time. If you've run the data generators you should have at least 1000 trip records.

```
EXPLAIN (ANALYZE) SELECT * FROM trips
WHERE completed_at IS NULL
ORDER BY completed_at;
```

The query uses a Sequential Scan of the trips table.

15. https://devcenter.heroku.com/articles/postgresql-indexes#b-trees-and-sorting